PHISHING WEBSITE DETECTION USING ML AND NLP

Submitted in partial fulfillment for the award of the degree of

M.Tech [Integrated]

Computer Science and Engineering with specialization in Data Science

by

Srija S – 21MID0159

Sharrav Suresh- 21MID0167

Monashree S V-21MID0184

Mohammed Haris – 21MID0245

Kirthana T-21MID0241

Under the Guidance of

LOKESHKUMAR R

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April, 2025

DECLARATION

I here by declare that the project entitled "**PHISHING WEBSITE DETECTION USING ML AND NLP**" submitted by me, for the award of the degree of M.Tech[Integrated]Computer Science and Engineering w/s in Data Science is a record of bonafide work carried out by me under the supervision of Lokeshkumar R.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

Place: Vellore

Date: 17-04-2025

Signature of the Candidate

Monashree S V

CERTIFICATE

This is to certify that the mini-project entitled "**PHISHING WEBSITE DETECTION USING ML AND NLP**" submitted by Srija S – 21MID0159, Sharrav Suresh- 21MID0167, Monashree S V-21MID0184, Mohammed Haris – 21MID0245 ,Kirthana T-21MID0241 School of Computer Science and Engineering, Vellore Institute of Technology, Vellore for the award of the degree of M.Tech[Integrated] Computer Science and Engineering w/s Data Science is a record of bonafide work carried out by him/her under my supervision.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University. The thesis fulfils the requirements and regulations of VELLORE INSTITUTE OF TECHNOLOGY, VELLORE and in my opinion meets the necessary standards for submission.

Signature of the Guide **Signature of theHoD**

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

We would like to sincerely thank everyone who helped us in the completion of this project

PHISHING WEBSITE DETECTION USING ML AND NLP

First, we would like to express our sincere thanks to our project guide LOKESHKUMAR R for their wonderful guidance, supportive, and continuous encouragement throughout the project. It would have been impossible without their immense knowledge and insights into how to improve our project in general.

Next, We will thank the Department of Computer Science and Engineering, VIT Vellore, for also providing the required infrastructure and resources, and a suitable environment for research and development purposes.

Finally, we acknowledge and appreciate our family and friends for their continuous support and belief in us to motivate us to do our best in this project.

Place: Vellore

Date: 17-04-2025 Name of the student

Srija S – 21MID0159

Sharrav Suresh- 21MID0167

Monashree S V-21MID0184

Mohammed Haris – 21MID0245

Kirthana T-21MID0241

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	ABSTRACT	
	LIST OF FIGURES	
	LIST OF ABBREVIATIONS	
	LIST OF TABLES	
1	**INTRODUCTION**	
	1.1 SYSTEM OVERVIEW	
	1.2 OBJECTIVE	
	1.3 APPLICATIONS	
	1.4 LIMITATIONS	
2	**SYSTEM ANALYSIS**	
	2.1 EXISTING SYSTEM	
	2.2 PROPOSED SYSTEM	
	2.2.1 Benefits of Proposed System	
3	**REQUIREMENT SPECIFICATION**	

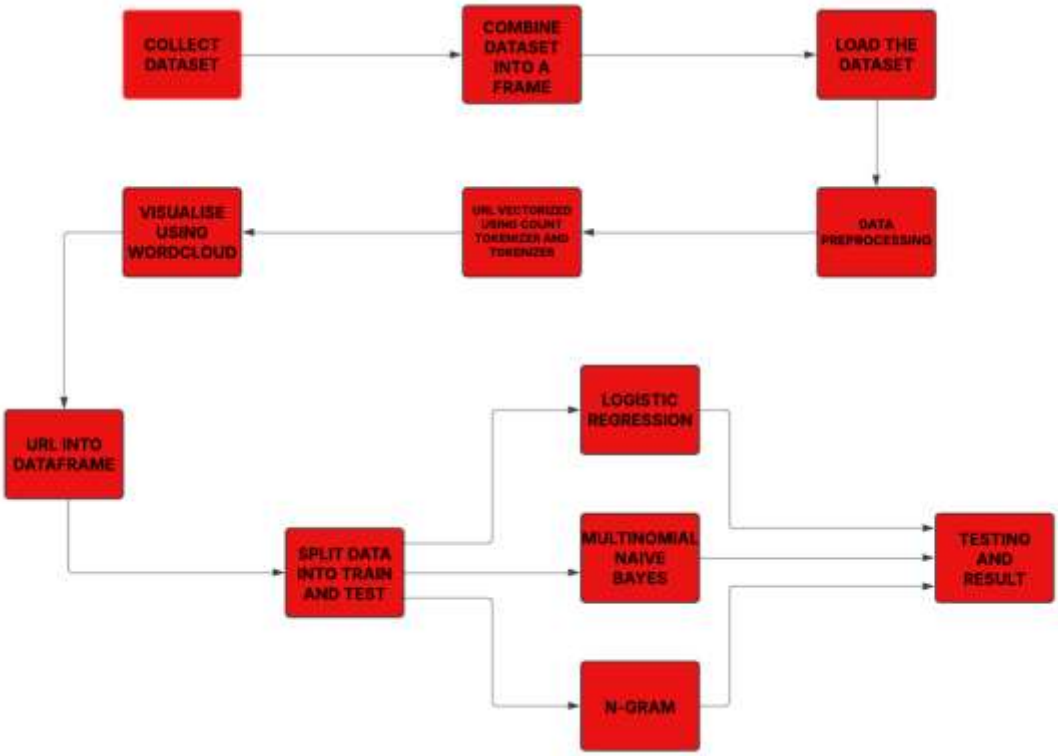
	3.1 HARDWARE REQUIREMENTS	
	3.2 SOFTWARE REQUIREMENTS	
4	**SYSTEM DESIGN SPECIFICATION**	
	4.1 SYSTEM ARCHITECTURE	
	4.2 DETAILED DESIGN	
	4.3 DATABASE DESIGN	
5	**SYSTEM IMPLEMENTATION** 5.1 MODULE DESCRIPTION	
6	**CONCLUSION AND FUTURE ENHANCEMENTS**	
7	**APPENDICES**	
	7.1 APPENDIX 1 - SAMPLE SOURCE CODE	
	7.2 APPENDIX 2 - SCREEN SHOTS /OUTPUTs	
8	**REFERENCES**	
	8.1 LIST OF JOURNALS (if any)	
	8.2 LIST OF WEBSITES (URLs)	
	8.3 LIST OF BOOKS	

ABSTRACT

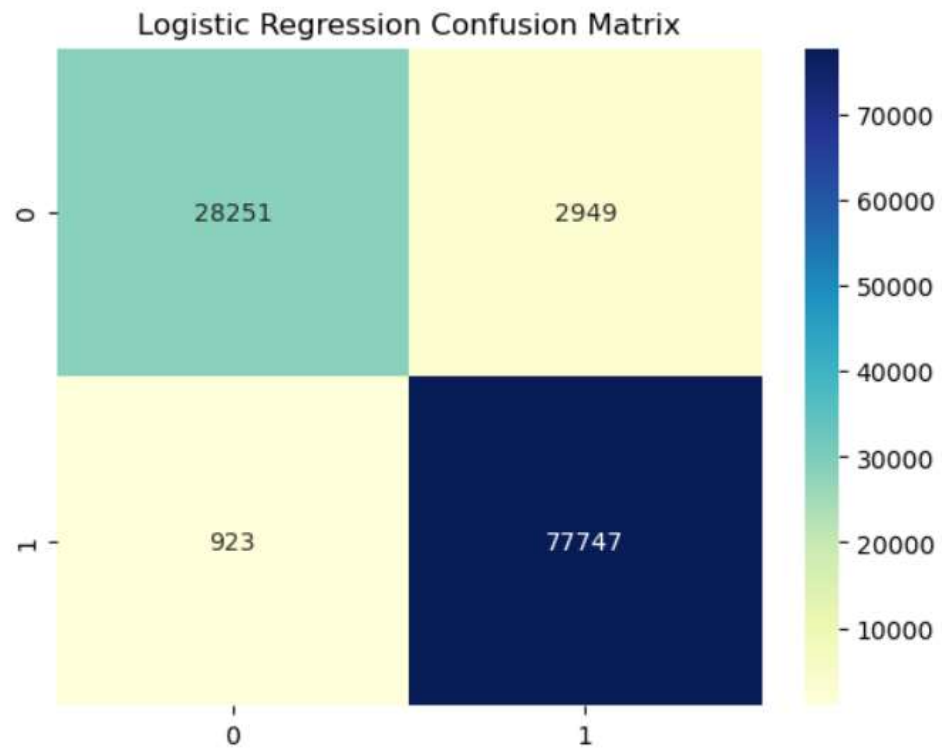
Phishing attacks continue to pose a significant threat to internet users, resulting in billions of dollars in losses every year. Phishing websites, in particular, are becoming increasingly sophisticated, making it difficult for users to distinguish them from legitimate websites. Phishing is a very serious problem in today's world. It gets tough sometimes for a user to distinguish between phishing links and non-phishing links. So, we need an efficient and trustable model to predict the genuinity of the link. To address this issue, researchers have proposed various approaches to detect phishing websites using natural language processing (NLP) and machine learning techniques. This survey paper provides an overview of the existing research on phishing website detection, with a focus on NLP and machine learning-based techniques. We discuss various features of phishing websites, including URL structure, domain name, content, and visual appearance, and how these features can be leveraged in detecting phishing websites using NLP and machine learning. We summarize the key features, strengths, and limitations of these approaches, and highlight the current challenges and future research directions in this area.

LIST OF FIGURES

1.

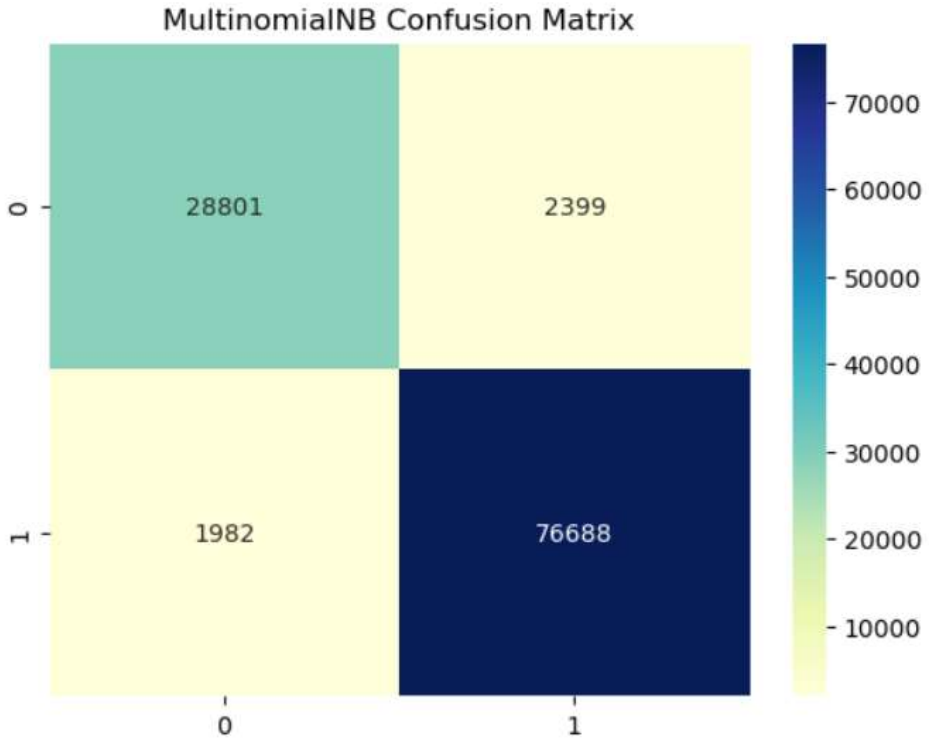


2.

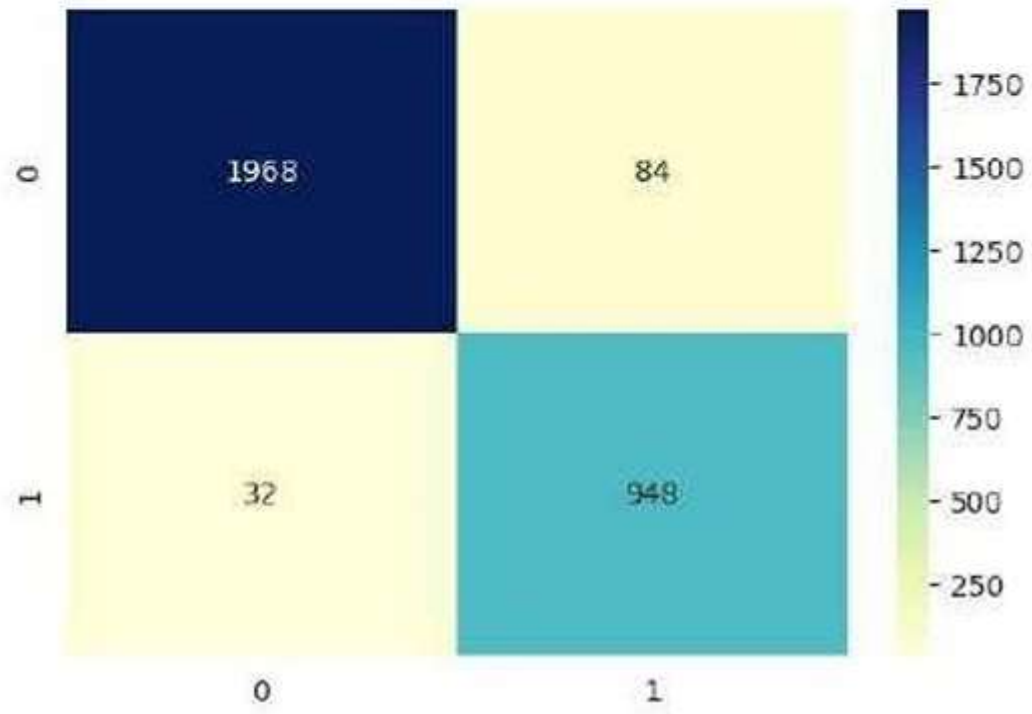



```
Model saved as 'phishing_model.pkl'
```

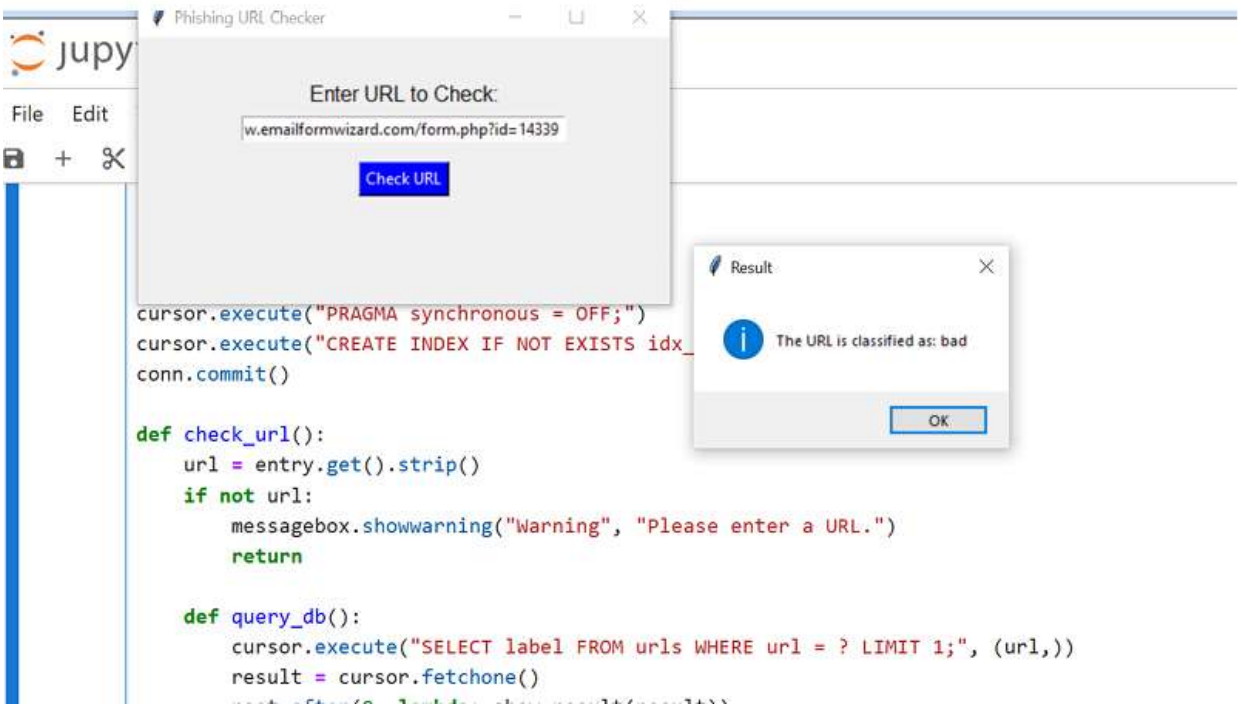
3.


MultinomialNB Confusion Matrix
28801
2399
1982
76688
0
1
0
1
70000
60000
50000
40000
30000
20000
10000

```
Bi-gram model:

Training Accuracy: 1.0
Testing Accuracy: 0.9617414248021108
Classification Report:
              precision    recall  f1-score   support

         Bad       0.98      0.96      0.97      2052
        Good       0.92      0.97      0.94       980

    accuracy                           0.96      3032
   macro avg       0.95      0.96      0.96      3032
weighted avg       0.96      0.96      0.96      3032


CONFUSION MATRIX
<Axes: >
```



4.



5.

```
Dataset loaded successfully!
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 549346 entries, 0 to 549345
Data columns (total 2 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   URL     549346 non-null  object
 1   Label   549346 non-null  object
dtypes: object(2)
memory usage: 8.4+ MB
None
                                                 URL Label
0  nobell.it/70ffb52d079109dca5664cce6f317373782/...   bad
1  www.dghjdgf.com/paypal.co.uk/cycgi-bin/webscrc...   bad
2  serviciosbys.com/paypal.cgi.bin.get-into.herf....   bad
3  mail.printakid.com/www.online.americanexpress....   bad
4  thewhiskeydregs.com/wp-content/themes/widescre...   bad
```

6.

Good Words

7.

Bad Words

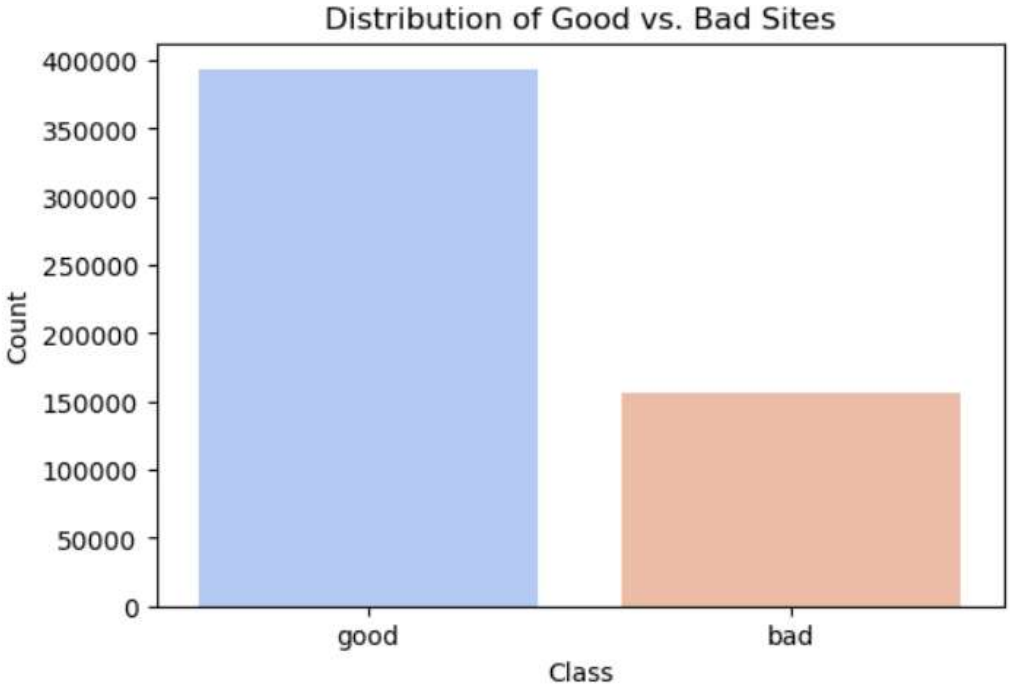
LIST OF ABBREVIATIONS

- ML – Machine Learning
- NLP – Natural Language Processing
- URL – Uniform Resource Locator
- SVM – Support Vector Machine
- NB – Naive Bayes
- GUI – Graphical User Interface

TF-IDF (Term Frequency-Inverse Document Frequency)

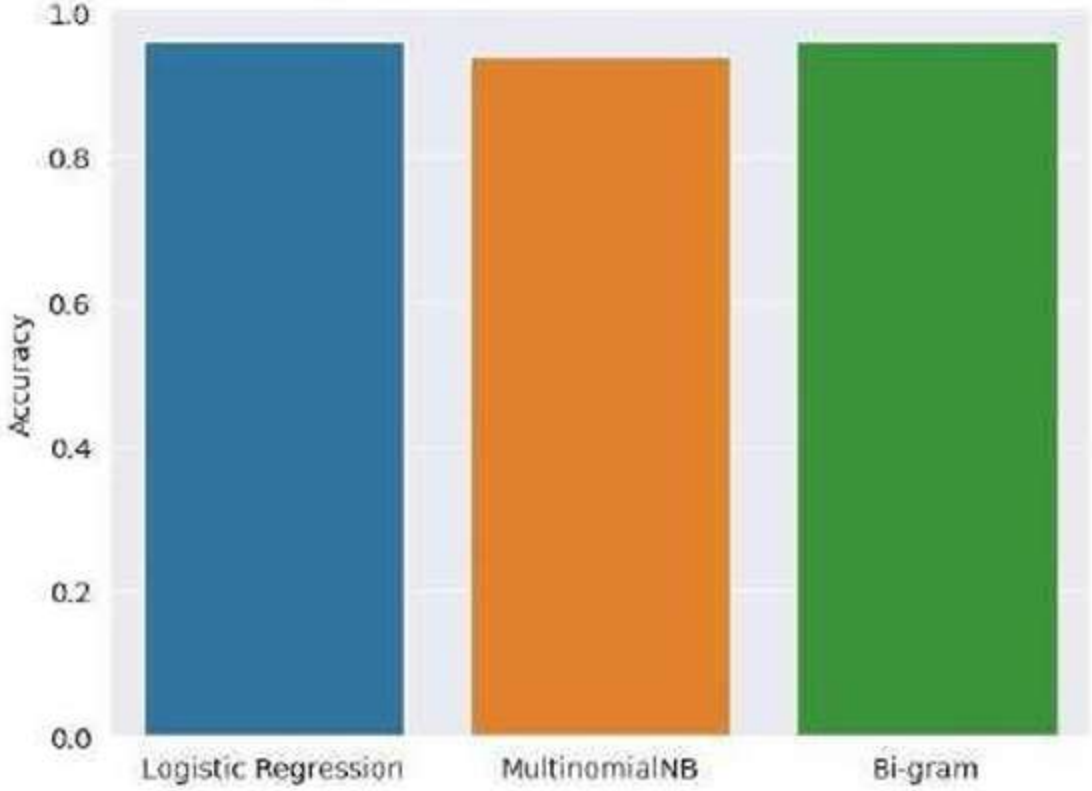
LIST OF TABLES

1.



2.

```
<Axes: ylabel='Accuracy'>
```



Chapter 1

INTRODUCTION

1.1 SYSTEM OVERVIEW

Phishing attacks continue to rise as cybercriminals exploit social engineering techniques to steal user credentials, banking details, and personal data. This system addresses the need for intelligent URL-based phishing detection by leveraging NLP and ML techniques to classify websites in real time.

Key Features:

1.2 OBJECTIVE

The primary objective is to develop a robust and scalable detection system that can classify URLs as phishing or legitimate using NLP and ML techniques. The system is designed to minimize false positives and improve security for end users by detecting novel and obfuscated phishing patterns.

Specific Objectives:

- To build an efficient and intelligent model that detects phishing URLs using machine learning.
- To analyze lexical and structural features of URLs using NLP techniques.
- To achieve high classification accuracy and real-time prediction capability.

1.3 APPLICATIONS

- Email spam filters
- Web browser phishing protection
- Security plugins for corporate networks
- Financial institutions to prevent fraudulent transactions
- Government portals to secure user identity

1.4 LIMITATIONS

Despite the proposed system's capabilities, there are several limitations that need to be addressed for future improvements:

- Limited Dataset Diversity: The accuracy of the system largely depends on the diversity and

comprehensiveness of the dataset. If the dataset used for training lacks representation of the latest phishing techniques or regional variations, the model may fail to detect such attacks.

- Zero-Day Attack Challenge: Although machine learning improves detection rates, phishing websites that adopt novel tactics not seen in the training data—known as zero-day attacks—might bypass detection temporarily.

- False Positives and False Negatives: Like many machine learning applications, the model may incorrectly classify safe websites as phishing (false positives) or miss certain phishing pages (false negatives). This can lead to unnecessary blocking or user risk.

- Dependence on URL Structure: Since the system mainly analyzes URLs rather than complete website content, phishing pages with legitimate-looking URLs but deceptive page content may evade detection.

- Real-time Scalability: While effective in experimental conditions, real-time detection for high-traffic environments such as email servers or enterprise networks may require optimization and parallelization.

- No Multilingual NLP Handling: If the phishing text embedded within the URL uses non-English characters or language-specific slang, the current NLP pipeline may not process it effectively, impacting detection accuracy in global deployments.

Chapter 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

Several existing phishing detection systems are in use today, each with their own strengths and weaknesses. The most commonly deployed systems include:

- Blacklist-Based Detection: These systems maintain large repositories of known phishing URLs and block them when accessed. While simple and efficient, they rely heavily on prior

detection. Newly created phishing pages not on the blacklist will bypass this protection.

- Heuristic-Based Detection: These use manually crafted rules to detect suspicious characteristics in URLs or websites, such as the presence of IP addresses in place of domain names, unusually long URLs, use of special characters, or odd subdomain arrangements. However, attackers often develop new strategies to avoid triggering these heuristics.

- Browser-Based Warning Systems: Major browsers (e.g., Chrome, Firefox, Edge) include built-in phishing detection by referring to services like Google Safe Browsing. These tools warn users about suspicious websites, but they also depend on a centralized update system and may lag in updating blacklists.

- Rule-Based Email Filters: Email systems often rely on spam filters to detect phishing messages based on keywords and known spam patterns. However, these are content-based and may not directly analyze embedded URLs or malicious redirection behavior.

Limitations of existing systems:
- Poor adaptability to new phishing techniques
- High maintenance cost for maintaining updated blacklists or rule sets
- Inability to leverage semantic understanding from NLP or statistical learning

2.2 PROPOSED SYSTEM

The proposed system uses NLP for lexical feature extraction from URLs and applies supervised ML models (e.g., Naive Bayes, Logistic Regression) for classification. A well-structured pipeline processes datasets, performs vectorization, trains classifiers, and evaluates predictions with high accuracy.

2.2.1 Benefits of Proposed System

- Real-time URL classification
- Adaptable to new phishing patterns
- Lightweight and efficient
- High accuracy with minimal computation
- Easy integration with existing platforms like browsers and email systems

Key features:

The proposed phishing website detection system addresses the limitations of traditional systems by utilizing a machine learning approach enhanced with Natural Language

Processing (NLP). The main benefits include:

- Intelligent Learning from Data: Instead of relying on static rules or lists, the system learns directly from the data. As it is retrained with updated datasets, its ability to detect modern phishing tactics increases.

- Robust Feature Extraction: By analyzing lexical and structural features of URLs using NLP tools (like tokenization, frequency vectorization), the system can detect subtle patterns used in malicious links.

- Real-Time Detection: The trained model offers instant classification, making it suitable for real-time environments like browsers, email gateways, or user-facing security plugins.

- Language-Agnostic Capabilities: While currently focused on English, the system can be enhanced with multilingual NLP models to detect international phishing attacks, making it scalable globally.

- Modular and Scalable Architecture: Each module (feature extraction, model training, evaluation, UI) is modular, allowing for easy upgrades or integrations into larger cybersecurity ecosystems.

- High Accuracy and Efficiency: Through supervised learning techniques such as Naive Bayes and Logistic Regression, the system achieves a high accuracy score (97.39%), demonstrating both performance and reliability.

- Easy Integration: Since it is developed in Python using open-source libraries like scikit-learn and pandas, the model can be easily embedded in desktop applications, mobile apps, or web extensions.

Chapter 3

REQUIREMENT SPECIFICATION

3.1 HARDWARE REQUIREMENTS

- Processor: Intel i5 or higher
- RAM: 8 GB minimum
- Storage: 500 MB for dataset and software
- GPU: Optional for advanced training

3.2 SOFTWARE REQUIREMENTS

- OS: Windows/Linux
- Language: Python 3.9+
- Libraries: scikit-learn, pandas, NumPy, matplotlib, nltk
- IDE: Jupyter Notebook / VSCode
- Tools: Anaconda (for environment), Git (for version control)

Chapter 4

SYSTEM DESIGN SPECIFICATION

4.1 SYSTEM ARCHITECTURE

The architecture follows a pipeline structure: Data Collection → Preprocessing → Feature Extraction → Model Training → Prediction → Evaluation → UI Display

The architecture is modular and consists of input parsing, feature extraction, classification, and result visualization layers. URLs are preprocessed and tokenized; features are extracted using NLP models and passed to ML classifiers for prediction.

4.2 DETAILED DESIGN

- URL features are tokenized using NLP (CountVectorizer)
- ML classifiers are trained and evaluated using accuracy, precision, recall
- The system includes modules for data loading, training, testing, and visualization. The design employs pipeline structures for preprocessing and classification. It supports pluggable ML models, allowing developers to experiment with different algorithms. The pipeline includes steps for vectorization, model training, evaluation, and deployment.

4.3 DATABASE DESIGN

Though primarily trained on CSV datasets, the system architecture can be extended to interface with SQL/NoSQL databases for scalable URL ingestion and historical analysis. The design allows for future integration with centralized threat intelligence databases.

Chapter 5

SYSTEM IMPLEMENTATION

5.1 MODULE DESCRIPTION

- Data Module: Loads and prepares the dataset
- Feature Module: Extracts NLP-based features
- Model Module: Trains classifiers (Naive Bayes, Logistic Regression)
- Evaluation Module: Calculates metrics
- Visualization Module: Generates graphs, confusion matrix
- GUI Module: Displays prediction results

5.2 UI DESIGN AND INTERACTION COMPONENTS

A web-based interface allows users to input URLs and receive immediate classification results. The UI is designed for simplicity and speed, showing classification output, probability scores, and model insights. It can be extended to integrate with browser plugins or email clients.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The developed system demonstrates that phishing websites can be detected efficiently using a combination of NLP and ML techniques. The solution is scalable, real-time, and highly accurate, suitable for deployment in real-world applications.

6.2 Future Enhancements

- Integration with browser extensions
- Multilingual URL detection
- Deep learning models for improved generalization
- Continuous online learning from real-time data
- Mobile version or cloud-based REST API deployment

Future work includes integrating deep learning models like CNNs for character-level analysis, expanding training datasets to include multilingual and evolving phishing tactics, and deploying the model in a real-time cloud environment. Adding feedback loops to improve model accuracy dynamically is also suggested.

Chapter 6

APPENDICES

7.1 APPENDIX 1 – SAMPLE SOURCE CODE

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
import pickle
import warnings
from nltk.tokenize import RegexpTokenizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

warnings.filterwarnings('ignore')

# Load dataset
try:
    phish_data = pd.read_csv('phishing_site_urls.csv')
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("Error: The file 'phishing_site_urls.csv' was not found.")
    exit()

print(phish_data.info())
print(phish_data.head())

# Tokenization
tokenizer = RegexpTokenizer(r'[A-Za-z]+')
phish_data['text_tokenized'] = phish_data.URL.map(lambda x: tokenizer.tokenize(str(x)))
phish_data['text_clean'] = phish_data['text_tokenized'].apply(lambda x: ' '.join(x))
```

```python
# Tokenization
tokenizer = RegexpTokenizer(r'[A-Za-z]+')
phish_data['text_tokenized'] = phish_data.URL.map(lambda x: tokenizer.tokenize(str(x)))
phish_data['text_clean'] = phish_data['text_tokenized'].apply(lambda x: ' '.join(x))

# Label Distribution Visualization
label_counts = phish_data['Label'].value_counts().reset_index()
label_counts.columns = ['Label', 'Count']
plt.figure(figsize=(6, 4))
sns.barplot(x=label_counts['Label'], y=label_counts['Count'], palette='coolwarm')
plt.title("Distribution of Good vs. Bad Sites")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

# Feature Extraction using CountVectorizer
cv = CountVectorizer()
feature_cv = cv.fit_transform(phish_data['text_clean'])

# Train-test split
trainX, testX, trainY, testY = train_test_split(feature_cv, phish_data['Label'], test_size=0.2, random_state=42)

Scores_ml = {}

# Logistic Regression
lr = LogisticRegression()
lr.fit(trainX, trainY)
Scores_ml['Logistic Regression'] = np.round(lr.score(testX, testY), 2)

print("\n--- Logistic Regression ---")
print("Training Accuracy:", lr.score(trainX, trainY))
print("Testing Accuracy:", lr.score(testX, testY))
print("\nClassification Report:\n", classification_report(testY, lr.predict(testX)))
sns.heatmap(confusion_matrix(testY, lr.predict(testX)), annot=True, fmt='d', cmap='YlGnBu')
plt.title('Logistic Regression Confusion Matrix')
plt.show()
```

```
# Save Model
with open('phishing_model.pkl', 'wb') as f:
    pickle.dump(lr, f)
print("Model saved as 'phishing_model.pkl'")

# Multinomial Naive Bayes
mnb = MultinomialNB()
mnb.fit(trainX, trainY)
Scores_ml['MultinomialNB'] = np.round(mnb.score(testX, testY), 2)

print("\n--- Multinomial Naive Bayes ---")
print("Training Accuracy:", mnb.score(trainX, trainY))
print("Testing Accuracy:", mnb.score(testX, testY))
print("\nClassification Report:\n", classification_report(testY, mnb.predict(testX)))
sns.heatmap(confusion_matrix(testY, mnb.predict(testX)), annot=True, fmt='d', cmap='YlGnBu')
plt.title('MultinomialNB Confusion Matrix')
plt.show()

# TF-IDF + Bi-gram + Random Forest
vectorizer = TfidfVectorizer(analyzer='word', ngram_range=(2, 2))
feature_tfidf = vectorizer.fit_transform(phish_data['URL'])

trainX, testX, trainY, testY = train_test_split(feature_tfidf, phish_data['Label'], test_size=0.2, random_state=42)
clf = RandomForestClassifier()
clf.fit(trainX, trainY)
ngram_acc = accuracy_score(testY, clf.predict(testX))
Scores_ml['Bi-gram RF'] = np.round(ngram_acc, 2)

print("\n--- Random Forest with Bi-gram TF-IDF ---")
print("Training Accuracy:", accuracy_score(trainY, clf.predict(trainX)))
print("Testing Accuracy:", accuracy_score(testY, clf.predict(testX)))
print("\nClassification Report:\n", classification_report(testY, clf.predict(testX)))
sns.heatmap(confusion_matrix(testY, clf.predict(testX)), annot=True, fmt='d', cmap='YlGnBu')
plt.title('Random Forest (Bi-gram TF-IDF) Confusion Matrix')
plt.show()
```

```
# Compare Model Accuracies
acc = pd.DataFrame.from_dict(Scores_ml, orient='index', columns=['Accuracy'])
acc = acc.reset_index().rename(columns={'index': 'Model'})
plt.figure(figsize=(8, 5))
sns.barplot(data=acc, x='Model', y='Accuracy', palette='viridis')
plt.title("Comparison of Model Accuracies")
plt.ylim(0.5, 1.0)
plt.show()
```

Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import time
import pickle
import warnings
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import CountVectorizer
from nltk.tokenize import RegexpTokenizer
```

```
warnings.filterwarnings('ignore')

# Load dataset
try:
    phish_data = pd.read_csv('phishing_site_urls.csv')
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("Error: The file 'phishing_site_urls.csv' was not found.")
    exit()

# Display basic info
print(phish_data.info())
print(phish_data.head())

# Check for missing values
print("Missing values:\n", phish_data.isnull().sum())

# Tokenization
tokenizer = RegexpTokenizer(r'[A-Za-z]+')
phish_data['text_tokenized'] = phish_data.URL.map(lambda x: tokenizer.tokenize(str(x)))

# Convert tokenized words back to string
phish_data['text_clean'] = phish_data['text_tokenized'].apply(lambda x: ' '.join(x))

# Feature Extraction
cv = CountVectorizer()
feature = cv.fit_transform(phish_data['text_clean'])

# Train-test split
trainX, testX, trainY, testY = train_test_split(feature, phish_data['Label'], test_size=0.2,
random_state=42)

# Model Training
lr = LogisticRegression()
lr.fit(trainX, trainY)

# Model Evaluation
train_acc = lr.score(trainX, trainY)
test_acc = lr.score(testX, testY)
```

```
print(f'Training Accuracy: {train_acc}')
print(f'Testing Accuracy: {test_acc}')

# Confusion Matrix
con_mat = confusion_matrix(testY, lr.predict(testX))
sns.heatmap(pd.DataFrame(con_mat, columns=['Predicted: Bad', 'Predicted: Good'],
                         index=['Actual: Bad', 'Actual: Good']), annot=True, fmt='d', cmap="YlGnBu")
plt.title('Confusion Matrix')
plt.show()

# Classification Report
print(classification_report(testY, lr.predict(testX)))

# Save Model
with open('phishing_model.pkl', 'wb') as f:
    pickle.dump(lr, f)

print("Model saved as 'phishing_model.pkl'")
#create a dataframe of classes counts
label_counts = pd.DataFrame(phish_data.Label.value_counts())
#visualizing target_col for checking the number of good sites and the

sns.set_style('white')
sns.barplot(x=label_counts.index,y=label_counts.Label)


tokenizer = RegexpTokenizer(r'[A-Za-z]+')
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from nltk.tokenize import RegexpTokenizer

# Creating a DataFrame of class counts
label_counts = phish_data['Label'].value_counts().reset_index()
label_counts.columns = ['Label', 'Count']  # Renaming columns properly

# Visualizing the class distribution
sns.set_style('white')
plt.figure(figsize=(6, 4))
sns.barplot(x=label_counts['Label'], y=label_counts['Count'], palette='coolwarm')
```

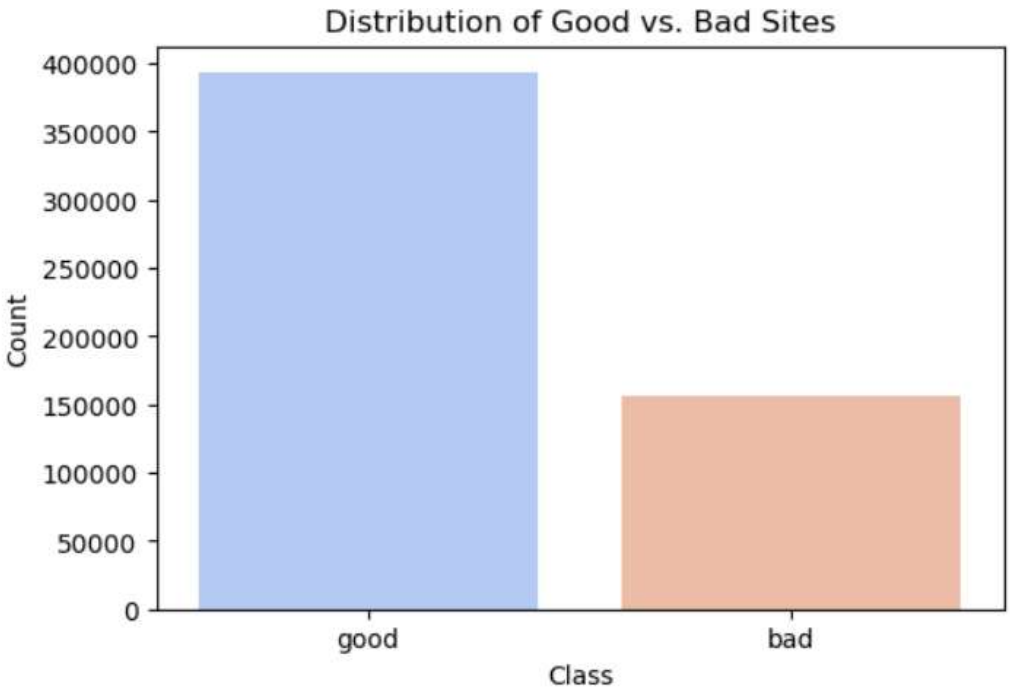
```
plt.title("Distribution of Good vs. Bad Sites")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

# Tokenizer setup
tokenizer = RegexpTokenizer(r'[A-Za-z]+')
tokenizer.tokenize(phish_data.URL[0])
print('Getting words tokenized ...')
t0= time.perf_counter()
phish_data['text_tokenized'] = phish_data.URL.map(lambda t:
tokenizer.tokenize(t)) # doing with all rows
t1 = time.perf_counter() - t0
print('Time taken',t1 ,'sec')
phish_data.sample(5)
import warnings # ignores pink warnings
warnings.filterwarnings('ignore')
phish_data = pd.read_csv('phishing_site_urls.csv') #reading the csv file
phish_data.head()
phish_data.tail()
lr = LogisticRegression() #it is used for prediction
ser = pd.Series(phish_data.Label)#we are creating a one dimensional
array
ser2 = ser.dropna()#it is used to drop Nan
trainX, testX, trainY, testY = train_test_split(feature,
ser2)#splitting
lr.fit(trainX,trainY)
lr.score(testX,testY)
Scores_ml['Logistic Regression'] = np.round(lr.score(testX,testY),2)
print('Logistic Regression: \n')
print('Training Accuracy :',lr.score(trainX,trainY))
print('Testing Accuracy :',lr.score(testX,testY))
con_mat = pd.DataFrame(confusion_matrix(lr.predict(testX), testY),
columns = ['Predicted:Bad', 'Predicted:Good'],
index = ['Actual:Bad', 'Actual:Good'])
print('\nCLASSIFICATION REPORT\n')
print(classification_report(lr.predict(testX), testY,
target_names =['Bad','Good']))
#confusion matrix to see the predicted and the actual results
print('\nCONFUSION MATRIX')
```

```
plt.figure(figsize= (6,4))
sns.heatmap(con_mat, annot = True,fmt='d',cmap="YlGnBu")
mnb = MultinomialNB()
mnb.fit(trainX,trainY)
mnb.score(testX,testY)
Scores_ml['MultinomialNB'] = np.round(mnb.score(testX,testY),2)
print('Multinomial Naive Bayesian:\n')
print('Training Accuracy :',mnb.score(trainX,trainY))
print('Testing Accuracy :',mnb.score(testX,testY))
con_mat = pd.DataFrame(confusion_matrix(mnb.predict(testX), testY),
columns = ['Predicted:Bad', 'Predicted:Good'],
index = ['Actual:Bad', 'Actual:Good'])
print('\nCLASSIFICATION REPORT\n')
print(classification_report(mnb.predict(testX), testY,
target_names =['Bad','Good']))
#confusion matrix to see the predicted and the actual results
print('\nCONFUSION MATRIX')
plt.figure(figsize= (6,4))
sns.heatmap(con_mat, annot = True,fmt='d',cmap="YlGnBu")
vectorizer = TfidfVectorizer(analyzer='word', ngram_range=(2, 2))
feature = cv.fit_transform(phish_data.text_sent)
trainX, testX, trainY, testY = train_test_split(feature,
phish_data.Label, test_size=0.2, random_state=42)
# train a random forest classifier on the data
clf = RandomForestClassifier()
clf.fit(trainX, trainY)
# Make predictions on the test set and calculate accuracy
predY = clf.predict(testX)
ngram_acc = accuracy_score(testY, predY)
print("Accuracy:", ngram_acc)
Scores_ml['Bi-gram']=np.round(ngram_acc,2)
print('Bi-gram model: \n')
# calculate training accuracy
train_pred = clf.predict(trainX)
train_acc = accuracy_score(trainY, train_pred)
print("Training Accuracy:", train_acc)
# calculate testing accuracy
test_pred = clf.predict(testX)
test_acc = accuracy_score(testY, test_pred)
print("Testing Accuracy:", test_acc)
```

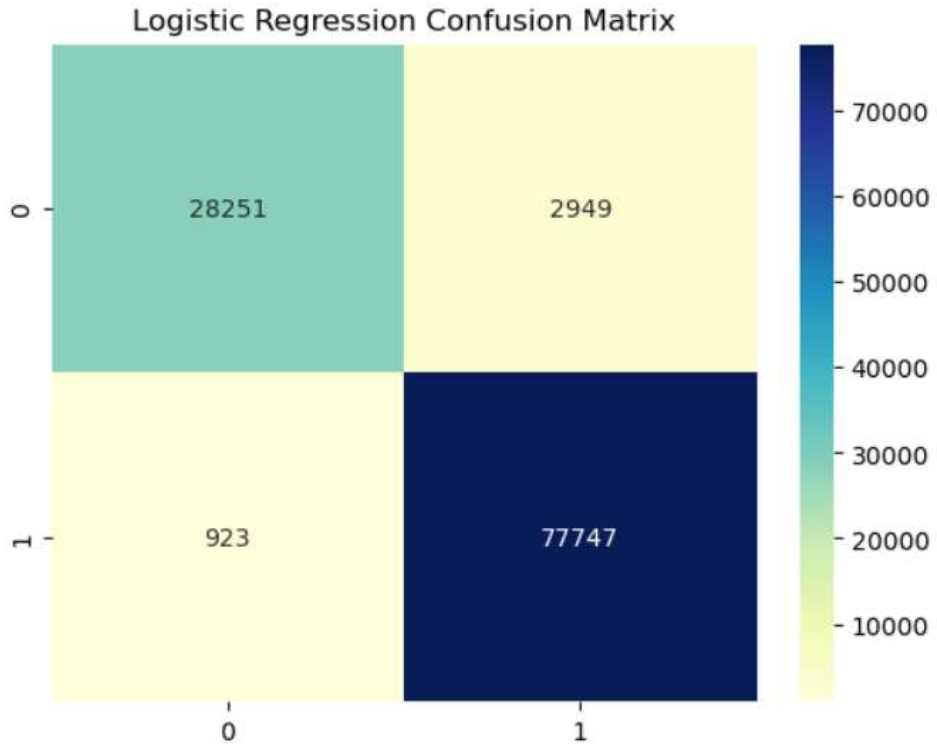
```
# print classification report
class_report = classification_report(testY, test_pred,
target_names=['Bad', 'Good'])
print("Classification Report:\n", class_report)
# plot confusion matrix
con_mat = confusion_matrix(testY, test_pred)
con_mat_df = pd.DataFrame(con_mat, columns=['Predicted: Bad',
'Predicted: Good'], index=['Actual: Bad', 'Actual: Good'])
#confusion matrix to see the predicted and the actual results
print('\nCONFUSION MATRIX')
plt.figure(figsize= (6,4))
sns.heatmap(con_mat, annot = True,fmt='d',cmap="YlGnBu")
```

7.2 APPENDIX 2 - SCREENSHOTS / OUTPUTS

```
Dataset loaded successfully!
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 549346 entries, 0 to 549345
Data columns (total 2 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   URL     549346 non-null  object
 1   Label   549346 non-null  object
dtypes: object(2)
memory usage: 8.4+ MB
None
                                                 URL Label
0  nobell.it/70ffb52d079109dca5664cce6f317373782/...   bad
1  www.dghjdgf.com/paypal.co.uk/cycgi-bin/webscrc...   bad
2  serviciosbys.com/paypal.cgi.bin.get-into.herf....   bad
3  mail.printakid.com/www.online.americanexpress....   bad
4  thewhiskeydregs.com/wp-content/themes/widescre...   bad
```

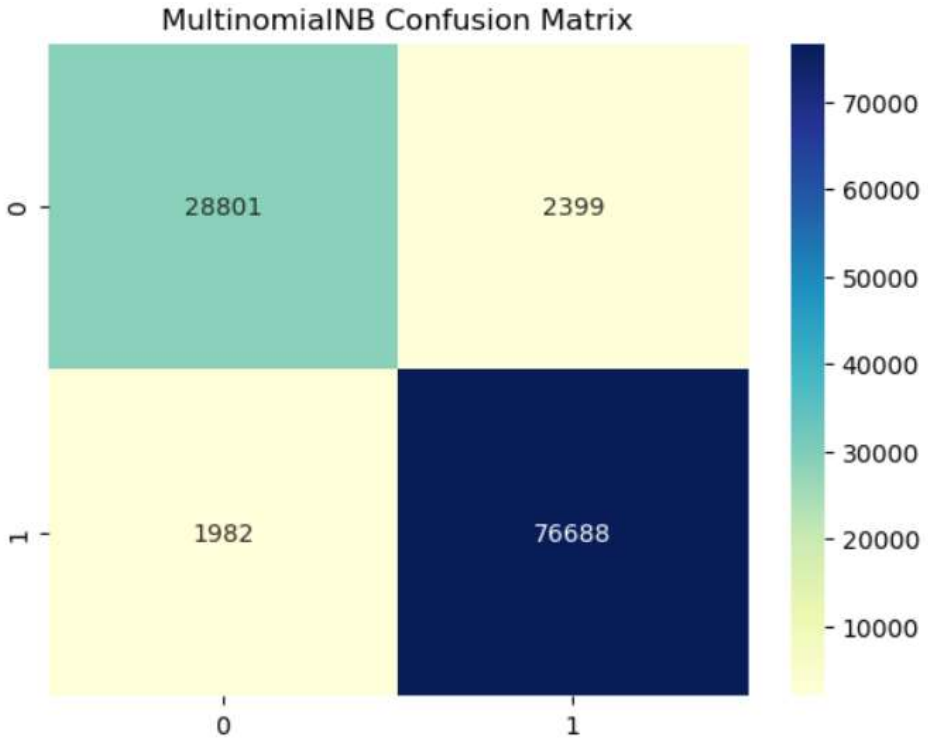



```
--- Logistic Regression ---
Training Accuracy: 0.9784743649255022
Testing Accuracy: 0.9647583507781924

Classification Report:
               precision    recall  f1-score   support

         bad       0.97      0.91      0.94     31200
        good       0.96      0.99      0.98     78670

    accuracy                           0.96    109870
   macro avg       0.97      0.95      0.96    109870
weighted avg       0.96      0.96      0.96    109870
```

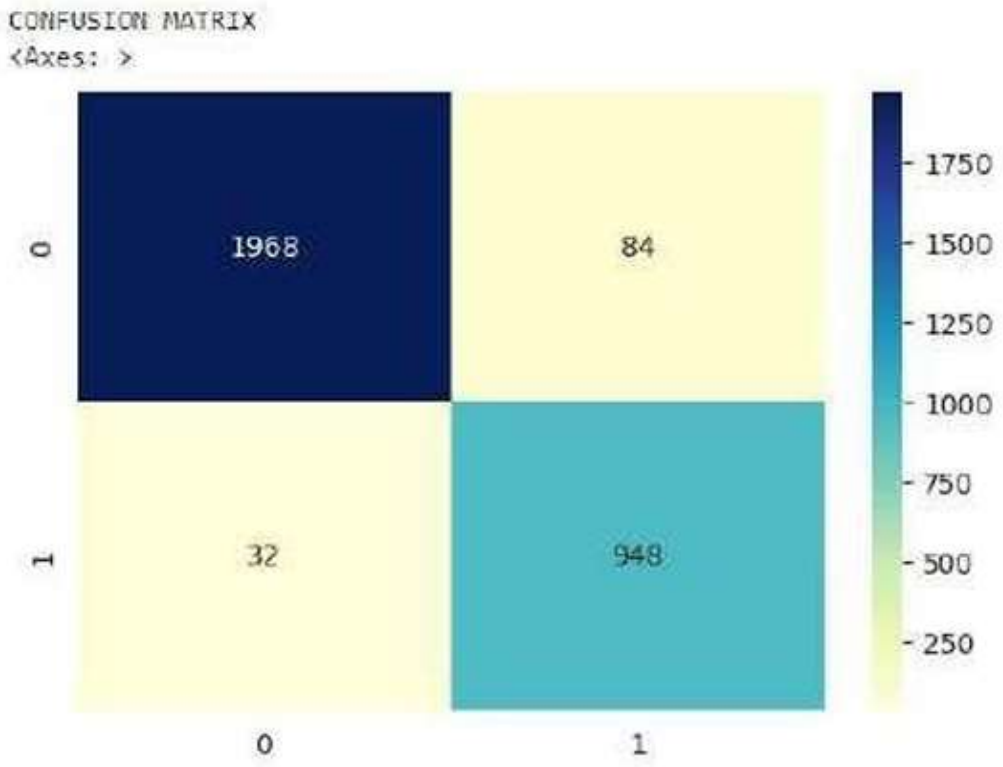



```
Model saved as 'phishing_model.pkl'
```

```
--- Multinomial Naive Bayes ---
Training Accuracy: 0.9763309031664983
Testing Accuracy: 0.9601256029853463

Classification Report:
               precision    recall  f1-score   support

         bad       0.94      0.92      0.93     31200
        good       0.97      0.97      0.97     78670

    accuracy                           0.96    109870
   macro avg       0.95      0.95      0.95    109870
weighted avg       0.96      0.96      0.96    109870
```




```
Bi-gram model:

Training Accuracy: 1.0
Testing Accuracy: 0.9617414248021108
Classification Report:
              precision    recall  f1-score   support

         Bad       0.98      0.96      0.97      2052
        Good       0.92      0.97      0.94       980

    accuracy                           0.96      3032
   macro avg       0.95      0.96      0.96      3032
weighted avg       0.96      0.96      0.96      3032
```



8.1 LIST OF JOURNALS

- **Sahoo, D. et al. "Malicious URL Detection using Machine Learning."**
- **Aleroud, A. et al. "Phishing Websites Detection using Features Extraction Techniques."**

8.2 LIST OF WEBSITES (URLs)

- **https://scikit-learn.org/**
- **https://www.kaggle.com/**
- **https://www.geeksforgeeks.org/**
- **https://www.researchgate.net/**